

System Driven Hardware Design

- Review of the Discrete Fourier Transform (DFT) -

International Master of Science

Prof. Dr.-Ing. Stephan Bannwarth

Agenda

- o **Review of the DFT and its properties**
- o DFT Scaling
- o Sinewaves and Noise
- o Averaging
- o Using the FFT in your project

Review of Signal Transformations

General Time- and Frequency representation:

$$x(t) \quad \circ \text{---} \bullet \quad X(f)$$

- Which transforms do you know?
- What can you say about the properties in the time and frequency domain?
- What's special at digital signals?

Time and Frequency Domain Properties

1. Two time domain properties determine the type of spectra:

- Continuous time or discrete time signal
 - Aperiodic or periodic signal
-
- ✓ **Continuous-time** signals have aperiodic spectra.
 - ✓ **Discrete-time** signals have periodic spectra.
 - ✓ **Periodic signals** have discrete spectra.
 - ✓ **Aperiodic finite energy signals** have continuous spectra.

2. Fourier Transformation of real signals:

- Even real part and odd imaginary part.



In the following: We focus on **real** and **time discrete** signals.

Quick DFT Derivation

DTFT:
$$X(\omega) = \sum_{n=-\infty}^{+\infty} x(n)e^{-j\omega n}$$



DFT:
$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j\frac{2\pi k}{N}n}$$

Relation between the sampling frequency f_s and the index number k of the DFT series:

$$\boxed{\frac{f_k}{f_s} = \frac{k}{N}}$$

FFT Command From Matlab

Matlab Help:

Fourier Transforms

R2018b

The Fourier transform is a mathematical formula that relates a signal sampled in time or space to the same signal sampled in frequency. In signal processing, the Fourier transform can reveal important characteristics of a signal, namely, its frequency components.

[Open Live Script](#)

The Fourier transform is defined for a vector x with n uniformly sampled points by

$$y_{k+1} = \sum_{j=0}^{n-1} \omega^{jk} x_{j+1}.$$

$\omega = e^{-2\pi i/n}$ is one of n complex roots of unity where i is the imaginary unit. For x and y , the indices j and k range from 0 to $n-1$.

$$Y_{k+1} = \sum_{j=0}^{n-1} x(j+1) e^{-i \frac{2\pi j}{n} k}$$

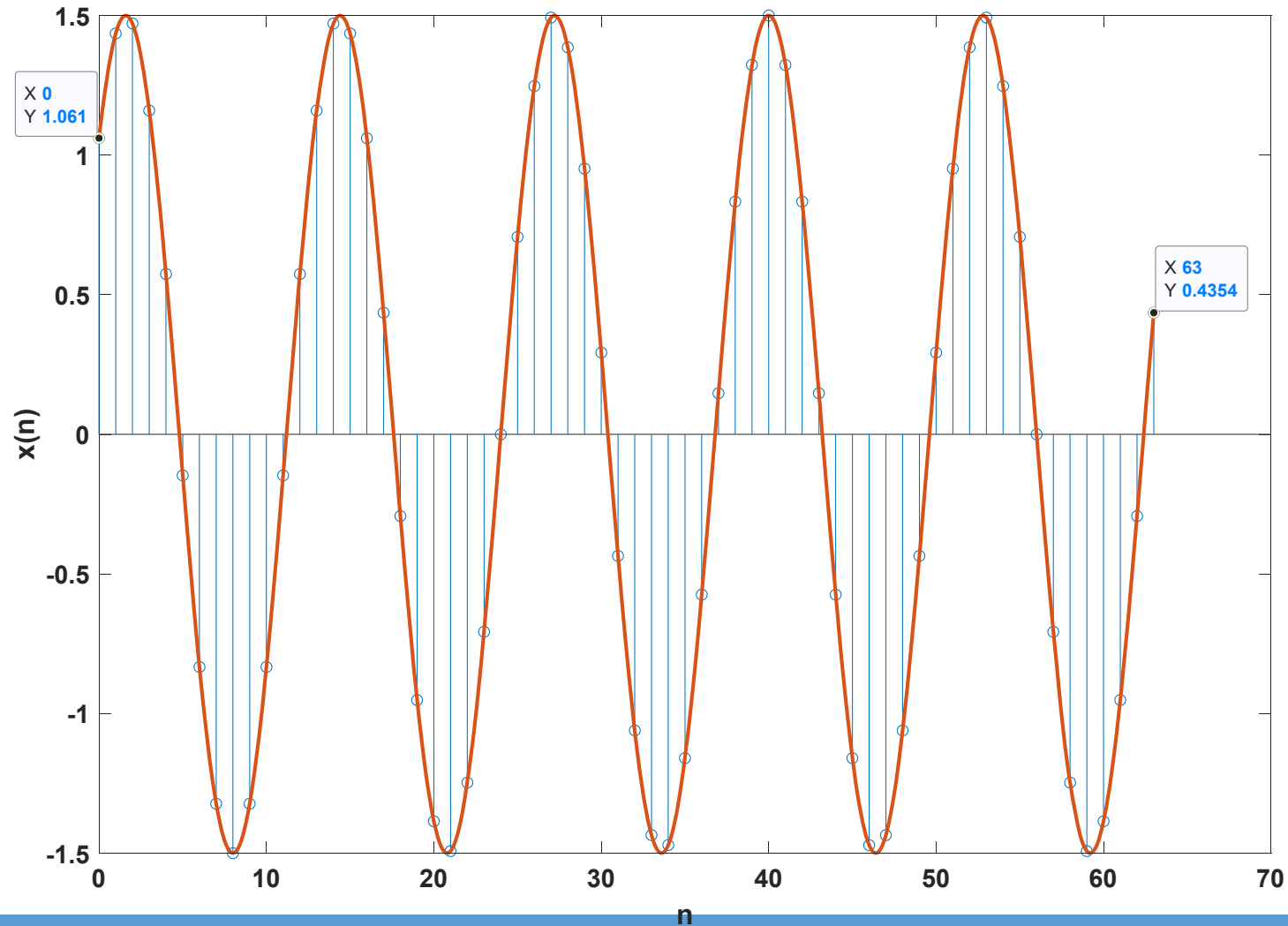
Sine Wave – Time Domain

Sinewave

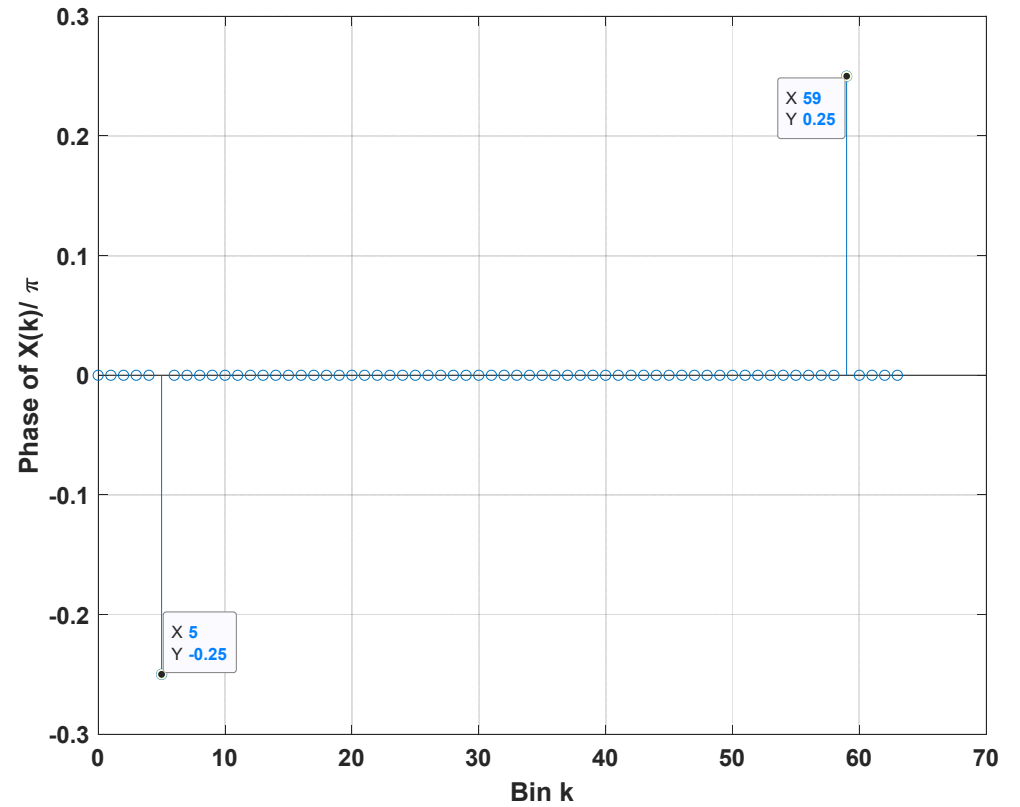
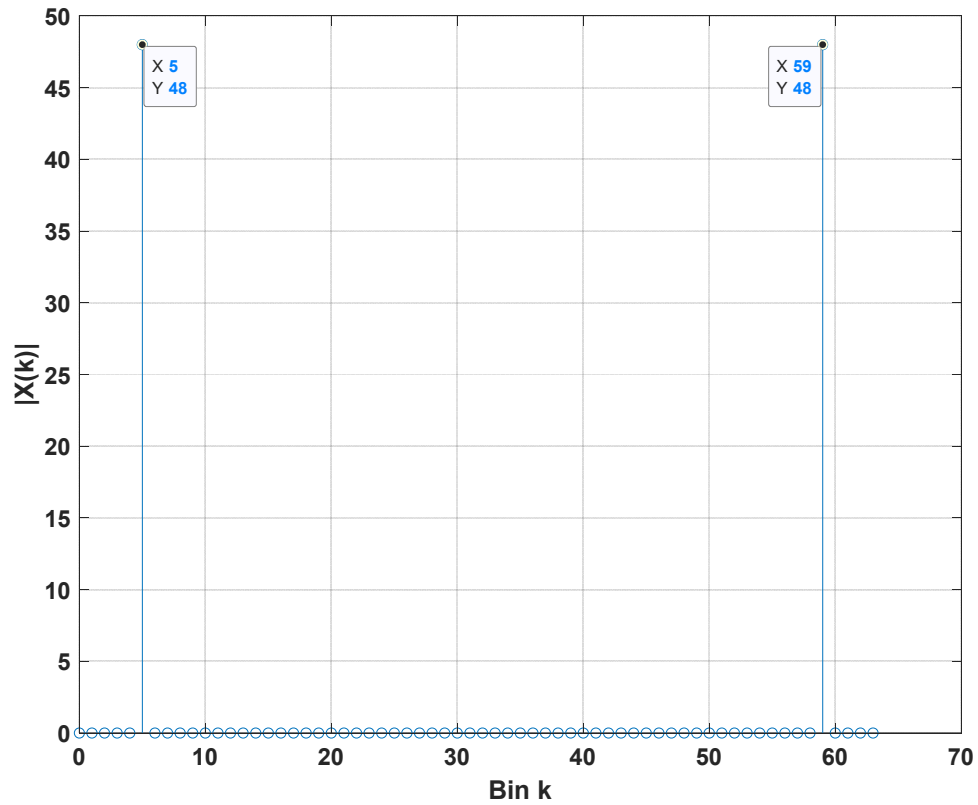
$f_1 = 20 \text{ Hz};$
 $\text{phase1} = -\pi/4$

Sampling and DFT

$f_s = 256 \text{ Hz};$
 $N = 64;$



Sine Wave – FFT in Matlab – no scaling



Agenda

- o Review of the DFT and its properties
- o **DFT Scaling**
- o Sinewaves and Noise
- o Averaging
- o Using the FFT in your project

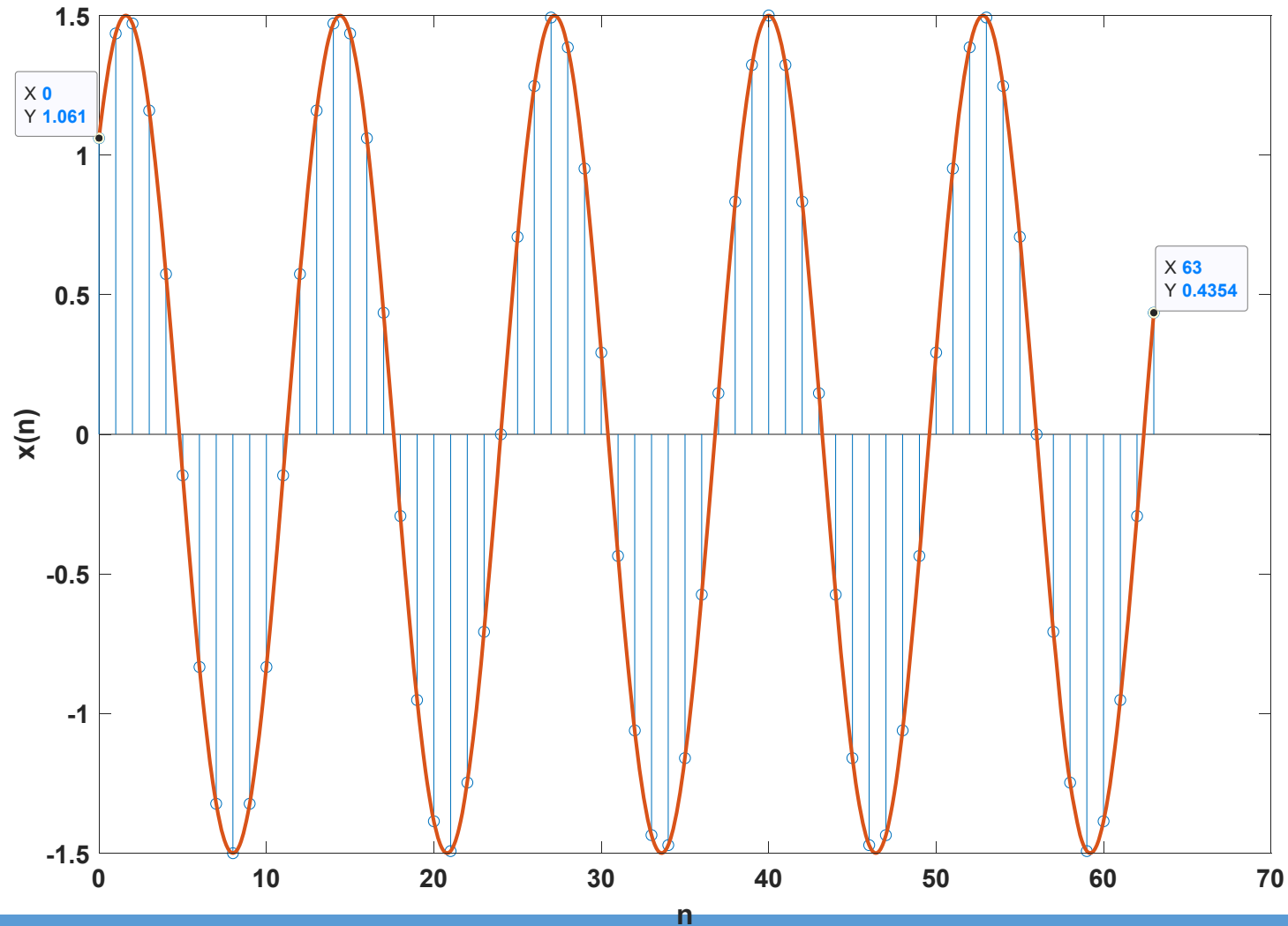
Sine Wave – Time Domain

Sinewave

$f_1 = 20 \text{ Hz};$
 $\text{phase1} = -\pi/4$

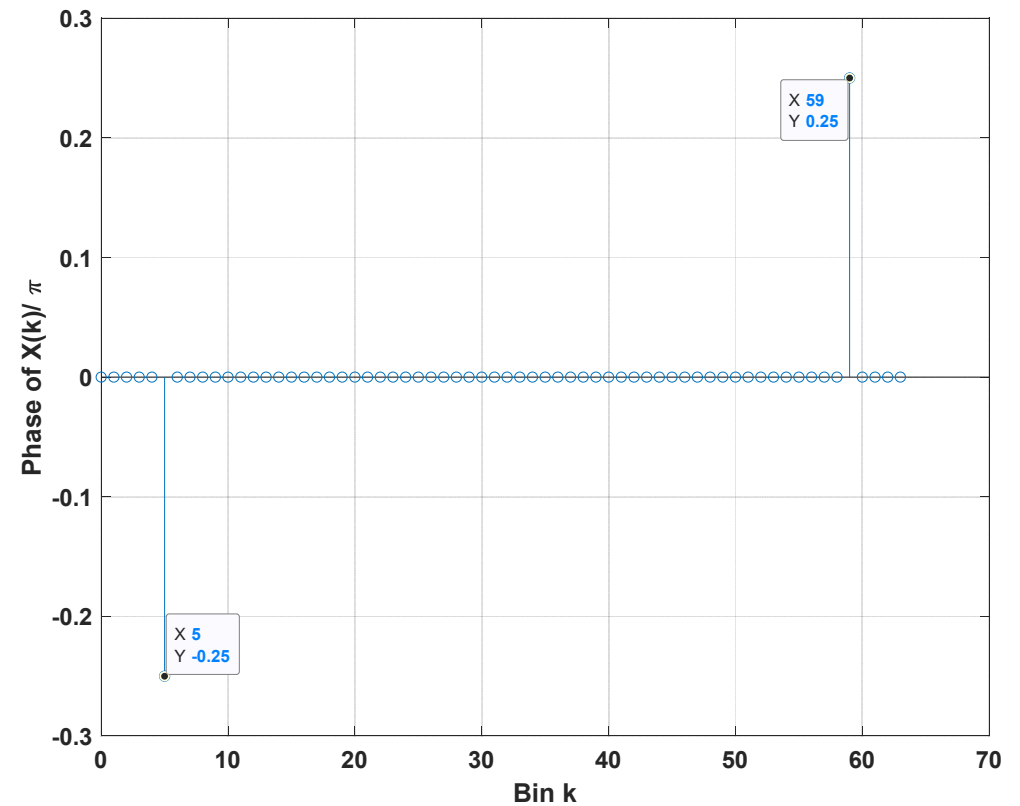
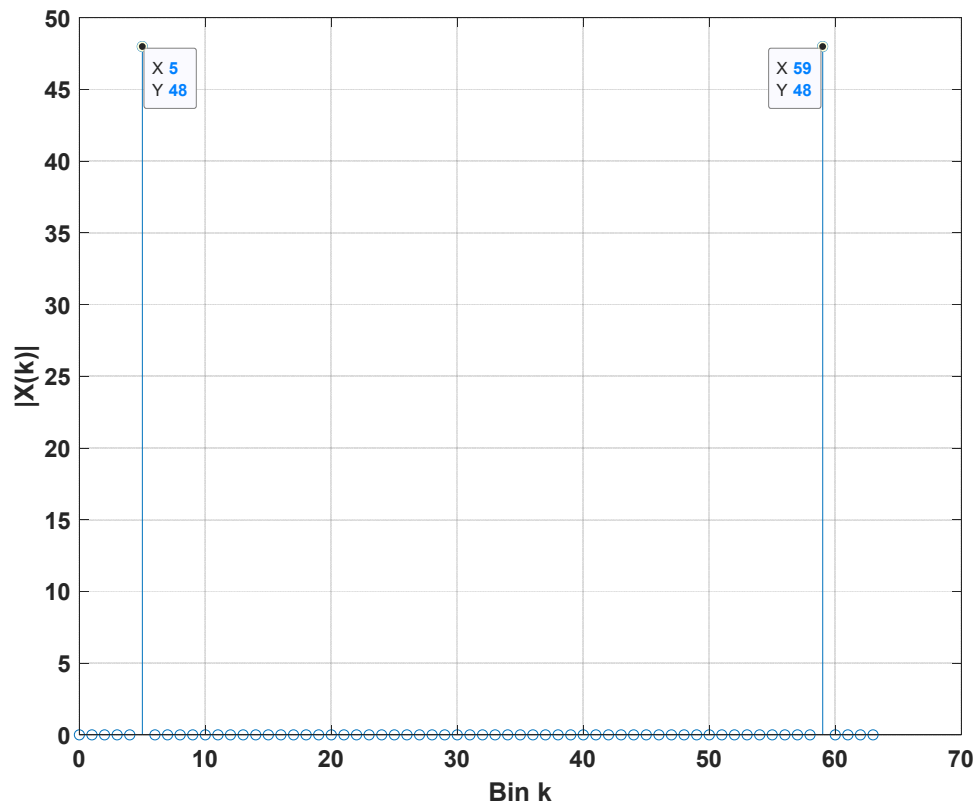
Sampling and DFT

$f_s = 256 \text{ Hz};$
 $N = 64;$



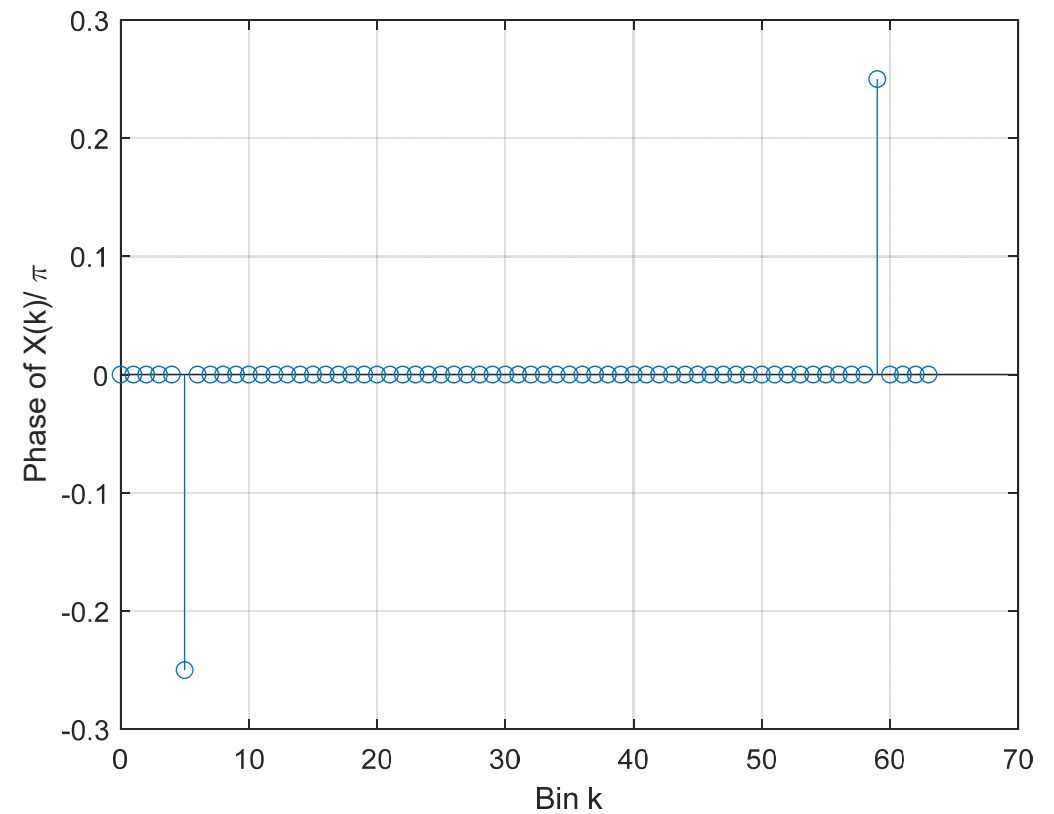
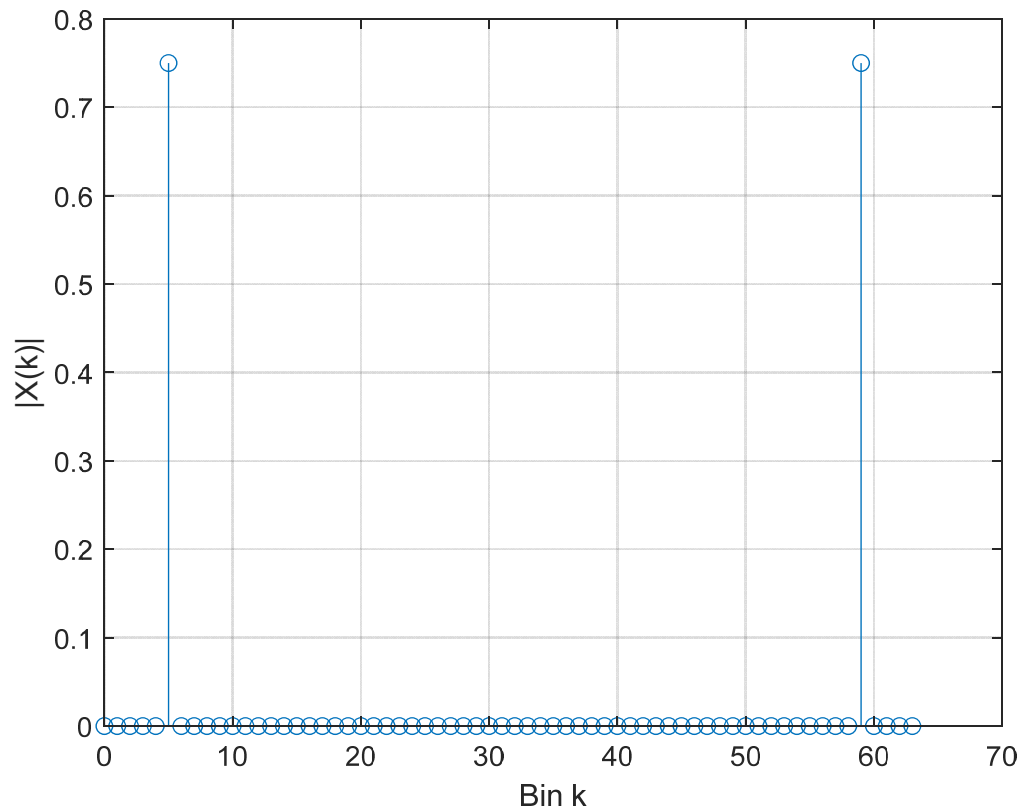
Sine Wave – FFT in Matlab – no scaling

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j\frac{2\pi k}{N}n}$$



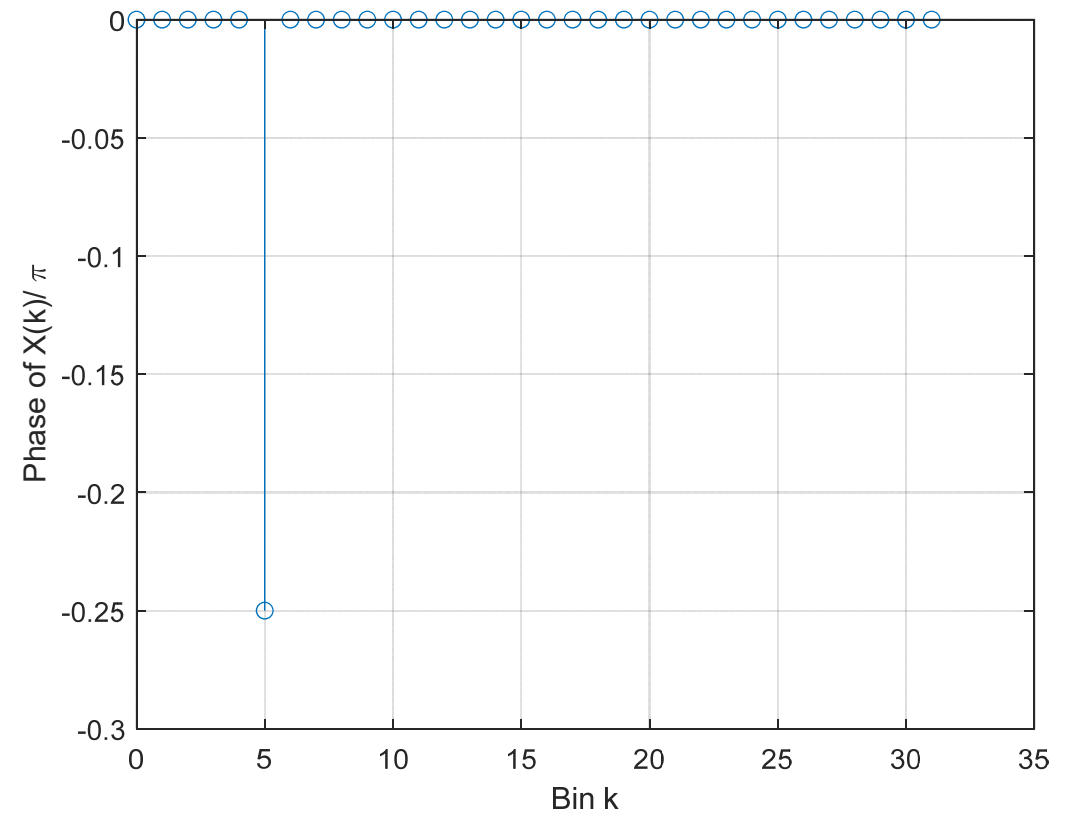
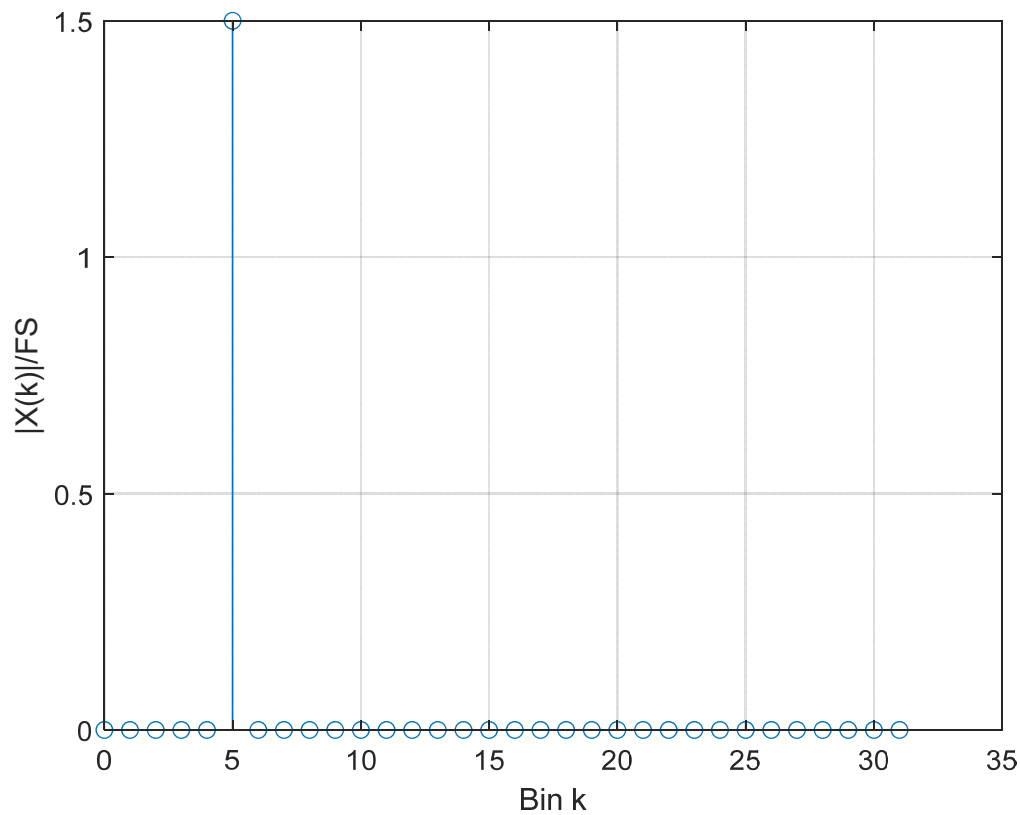
Sine Wave – FFT in Matlab – scaled to Amplitude

$$X(k) = \frac{1}{N} \sum_{n=0}^{N-1} x(n) e^{-j \frac{2\pi k}{N} n}$$

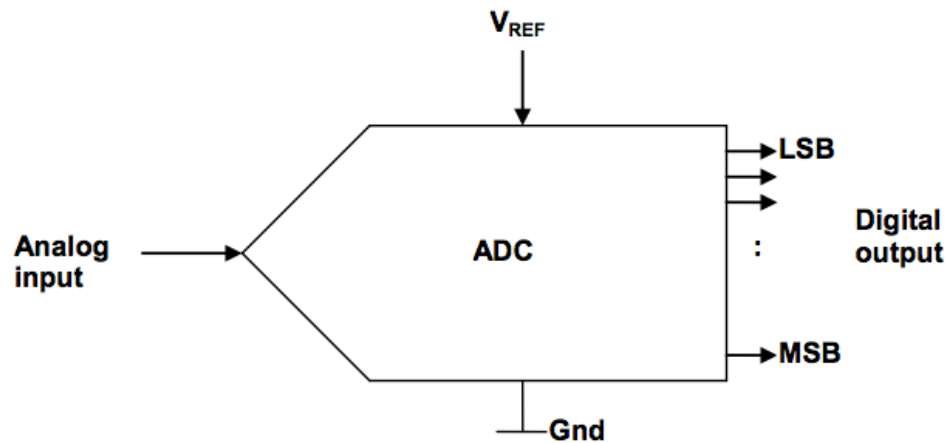


Sine Wave – FFT in Matlab – scaled to Amplitude – single sided spectra

$$X(k) = \frac{2}{N} \sum_{n=0}^{N-1} x(n) e^{-j \frac{2\pi k}{N} n}$$



Now: Think about the output of an ADC



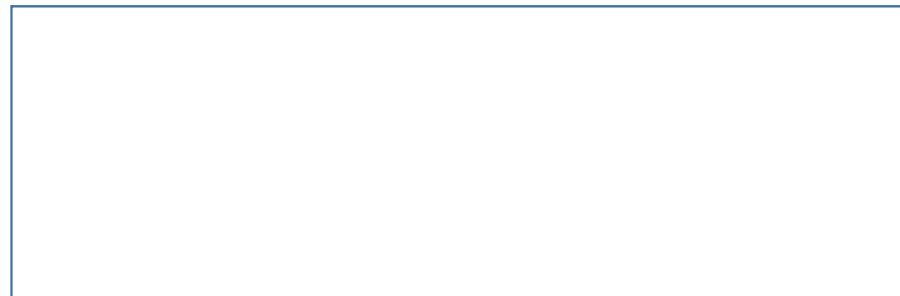
Assume

$n = 8$ Bit

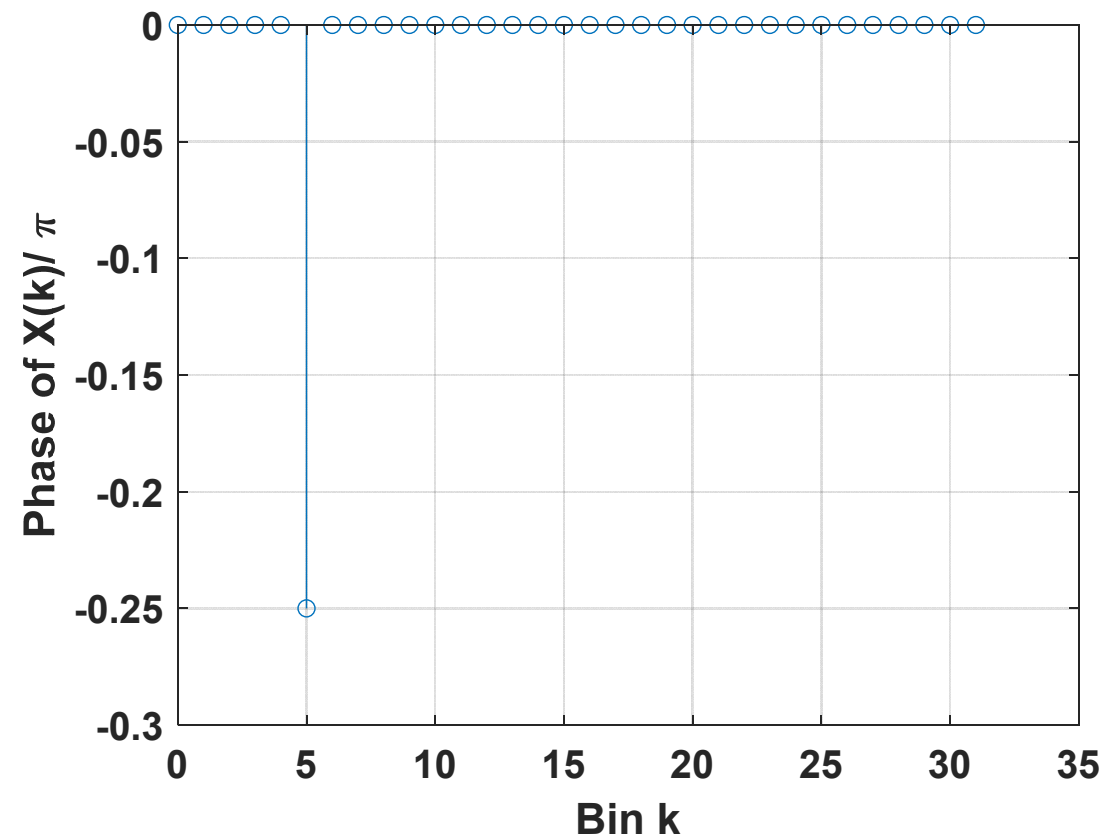
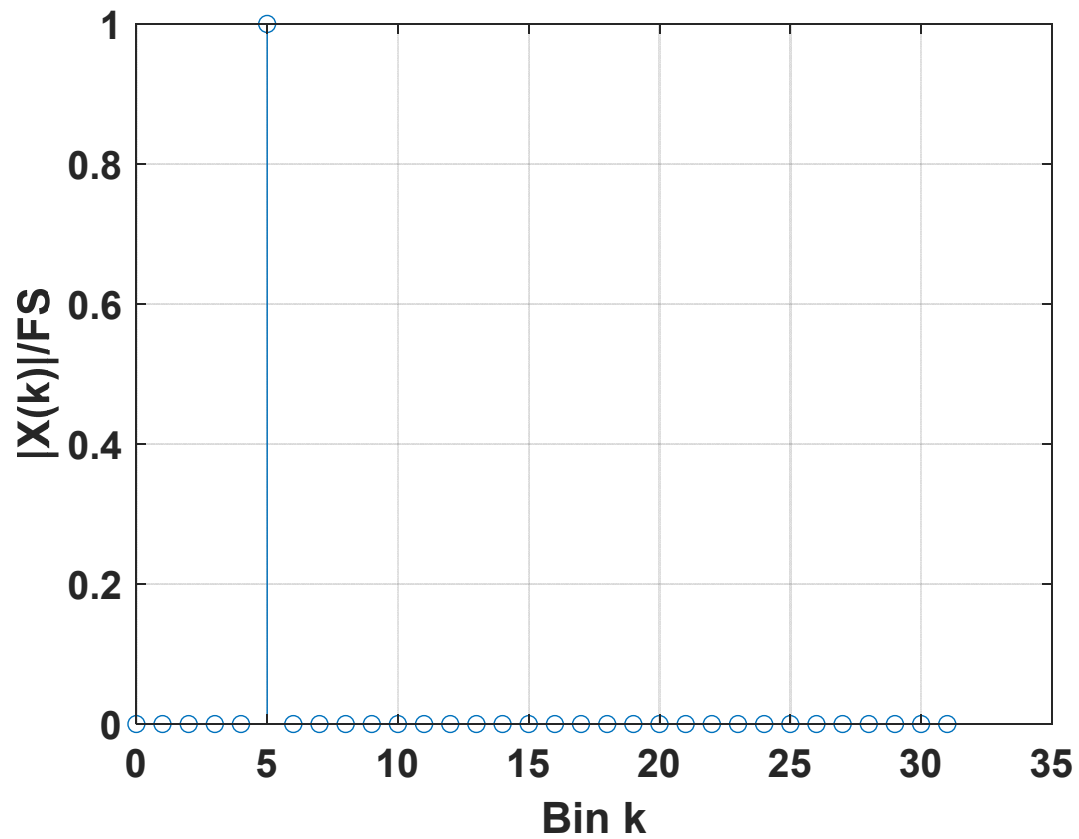
Applying a fullscale sinewave

Question

- How does a fullscale sinewave look like?
- How do we have to scale the DFT in order to get 1 in the single sided spectra, when a fullscale sinewave is applied?



Sine Wave – FFT in Matlab – scaled to fullscale – single sided spectra



Scaling – Aim and Factor

Aim of the Scaling: To get an information about the amplitude, energy or power of the spectral components of the sequence $X(k)$.

$$\text{DFT:} \quad X(k) = \sum_{n=0}^{N-1} x(n) e^{-j \frac{2\pi k}{N} n} \quad \text{IDFT:} \quad x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j \frac{2\pi k}{N} n}$$

Definition of the Fullscale Scaling (FS):

Assumptions:

In our case the spectra is symmetrical to DC. Therefore we only consider $k = 0 \dots N/2 - 1$

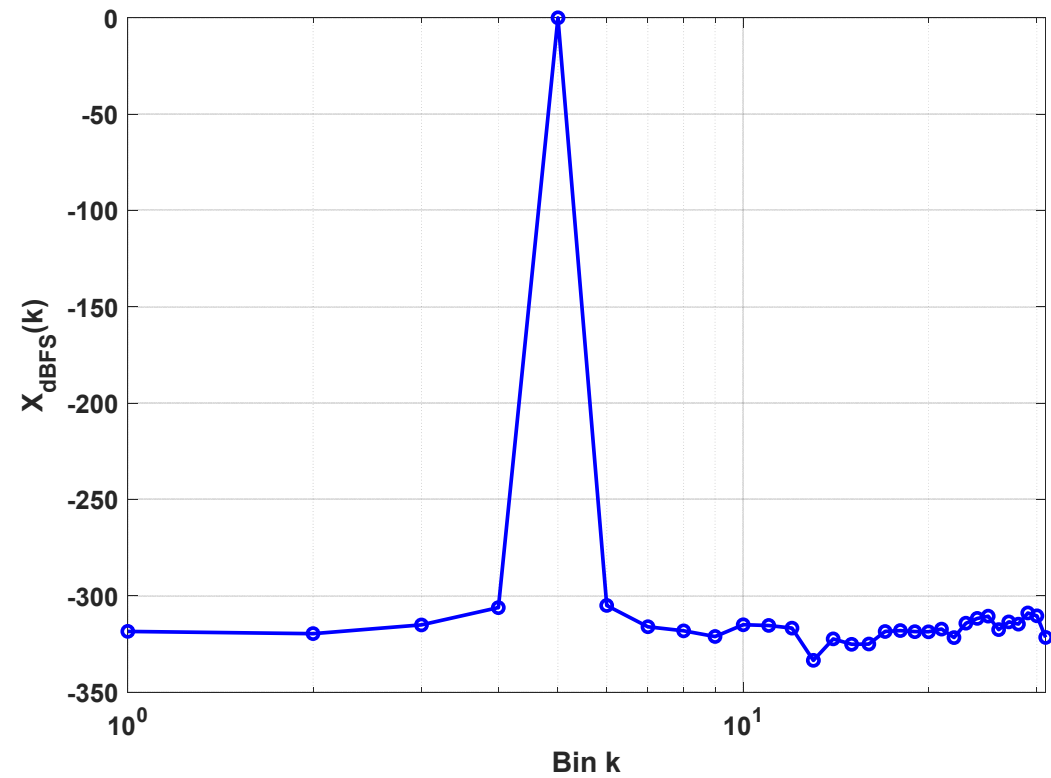
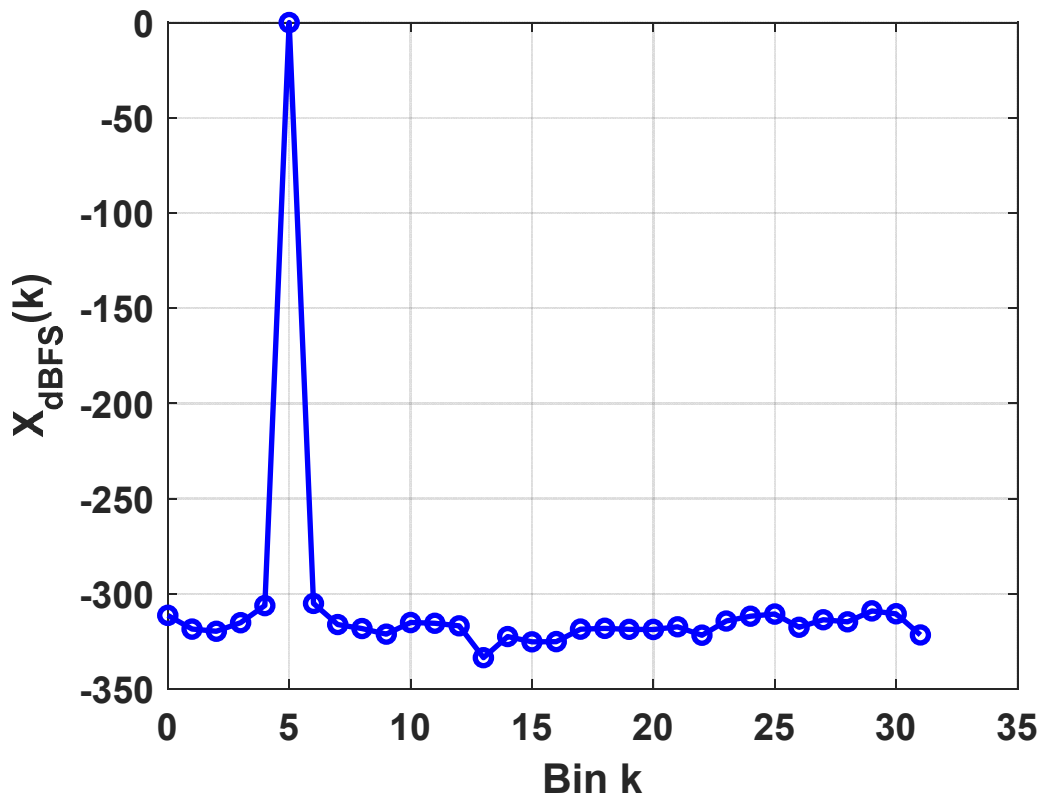
We scale to the fullscale of our sinewave input: **A0**

$$\frac{A_0 N}{2} = \frac{A_0 |W_1|}{2}$$

A_0 : Amplitude of the sinewave
 $|W_1|$: 1-norm of the window, here length N

Scaling – dB Full Scale (dBFS)

$$X_{dBFS}(k) = 20 \log_{10}(|X(k)|)$$

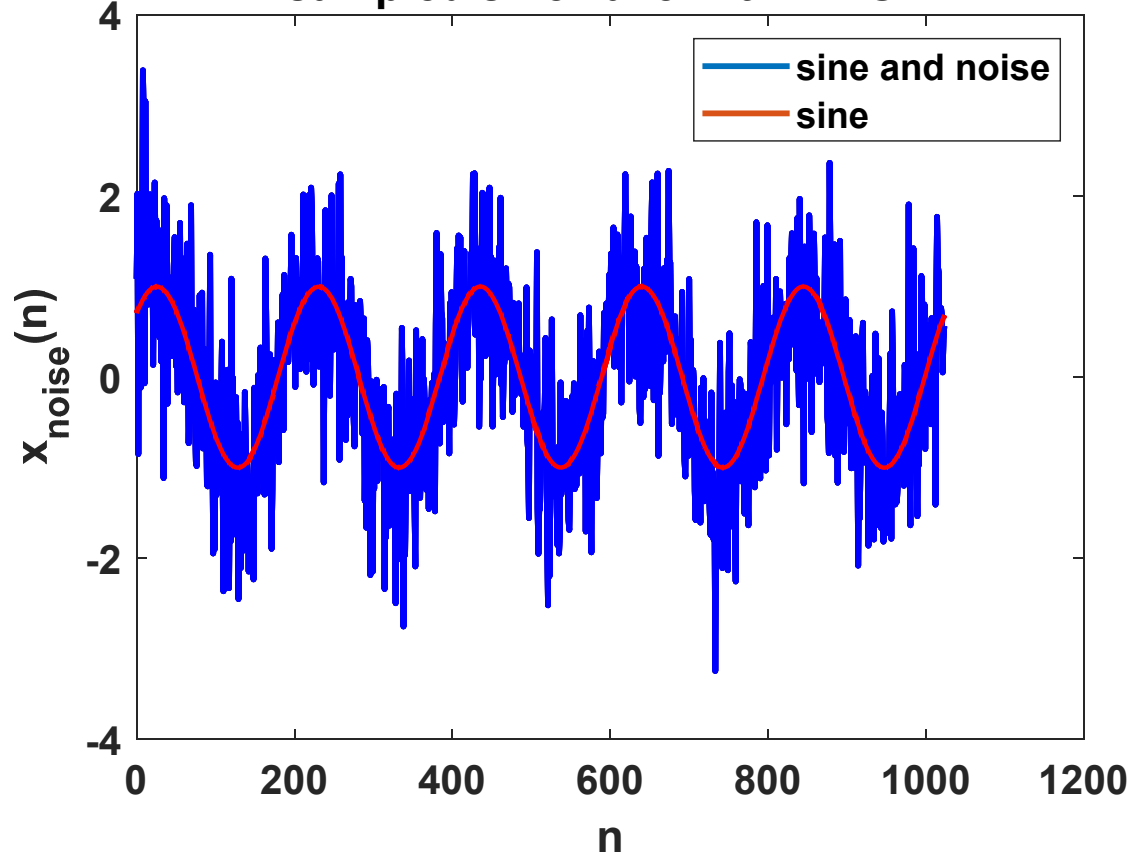


Agenda

- o Review of the DFT and its properties
- o DFT Scaling
- o **Sinewaves and Noise**
- o Averaging
- o Using the FFT in your project

Sinewave and additive white Gaussian noise (AWGN)

sampled sinewave with AWGN



$$x_{noise}(n) = x(n T_s) = A_0 \sin(\omega n T_s) + \text{noise}(n T_s)$$

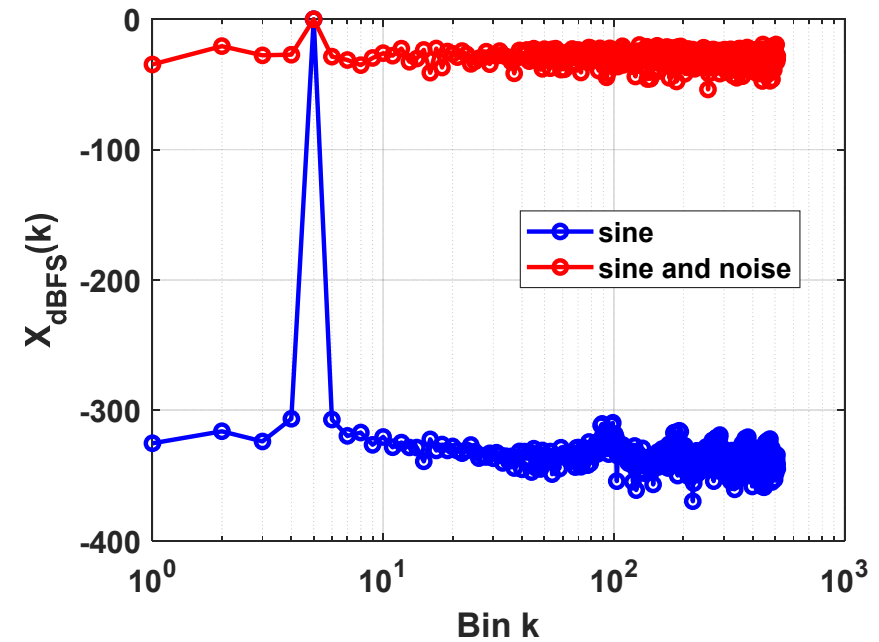
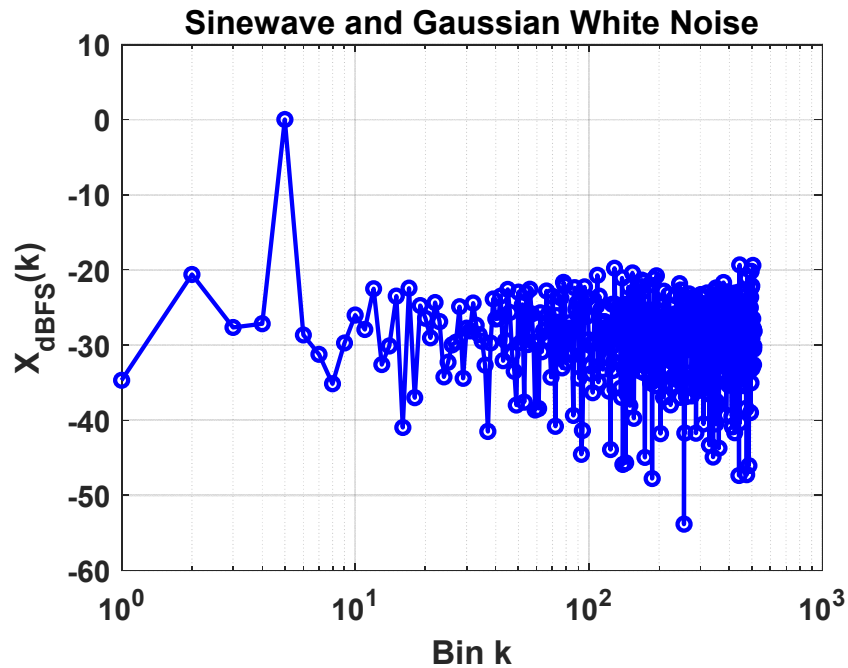
Questions

1. How do we compare different AWGNs?
2. Guess: Whats the noise power compared to the sinewave power?

In the this section the following parameters are used

- $A_0 = 1.0$
- $f_0 = 1.2500$
- $f_s = 256$
- $N = 1024$

Sine and Noise in the Frequency Domain

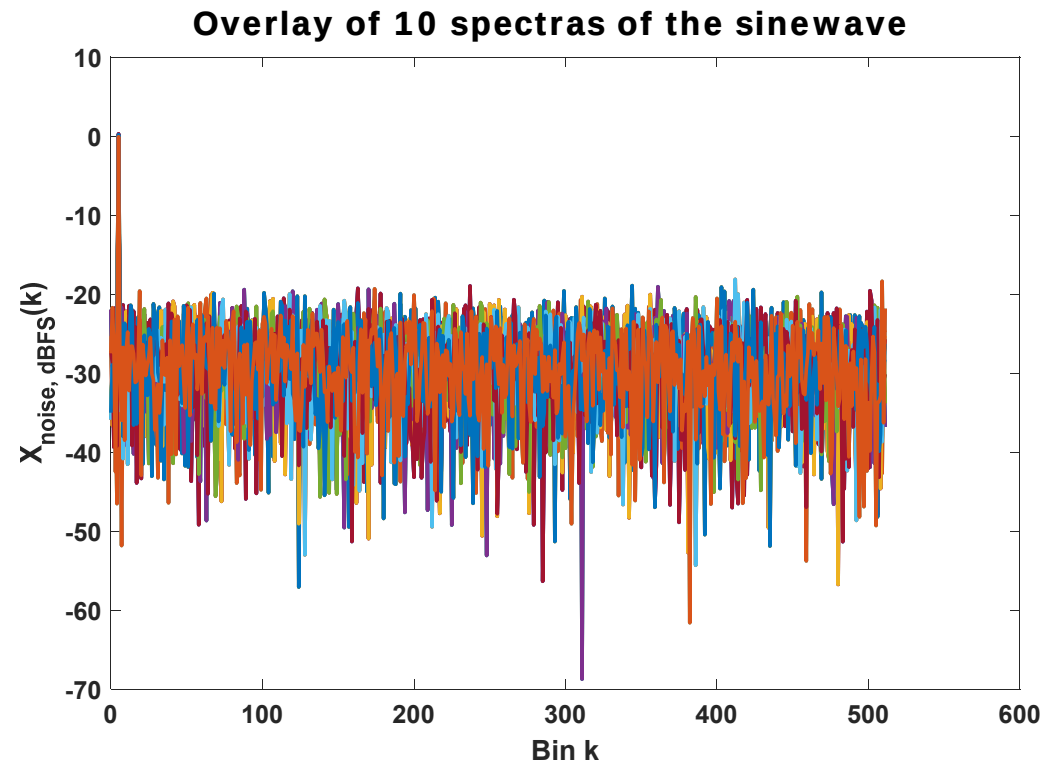
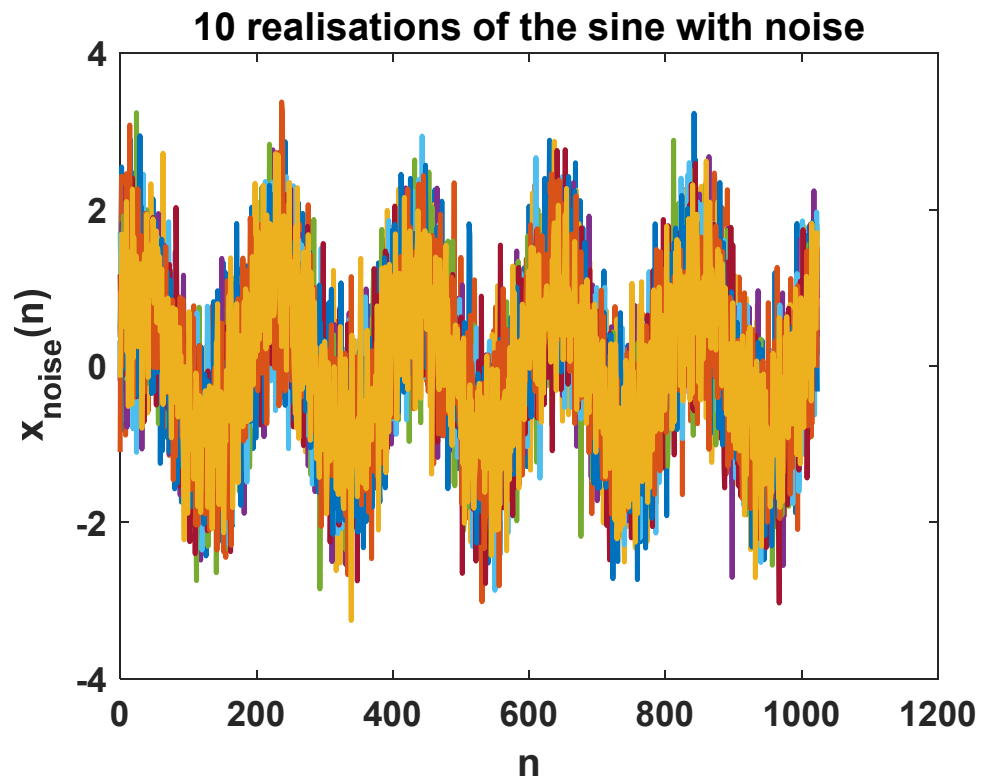


What's the effect of noise in the frequency domain / spectra?

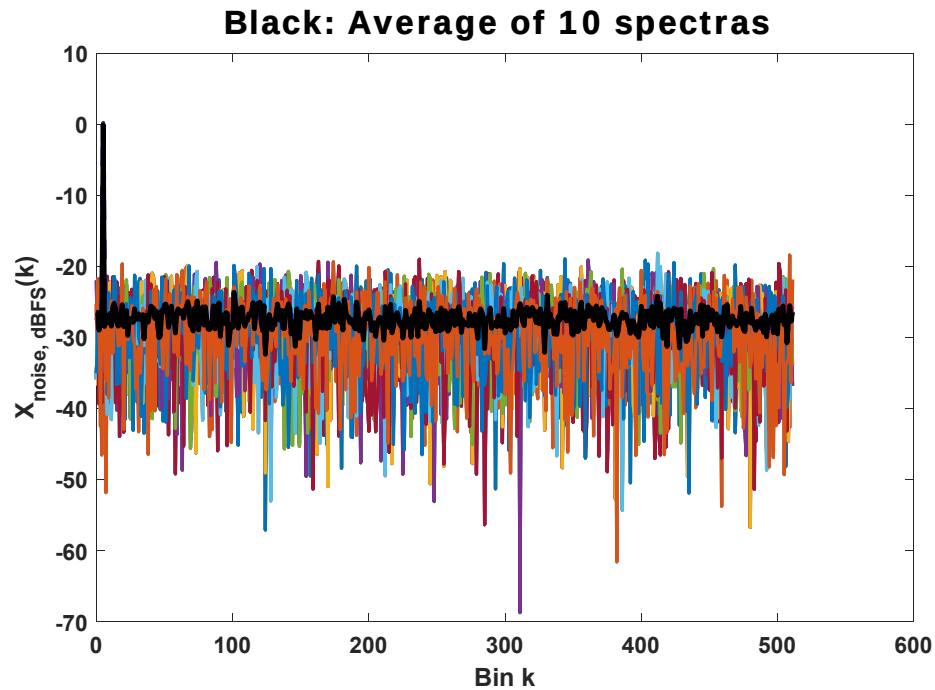
Agenda

- o Review of the DFT and its properties
- o DFT Scaling
- o Sinewaves and Noise
- o **Averaging**
- o Using the FFT in your project

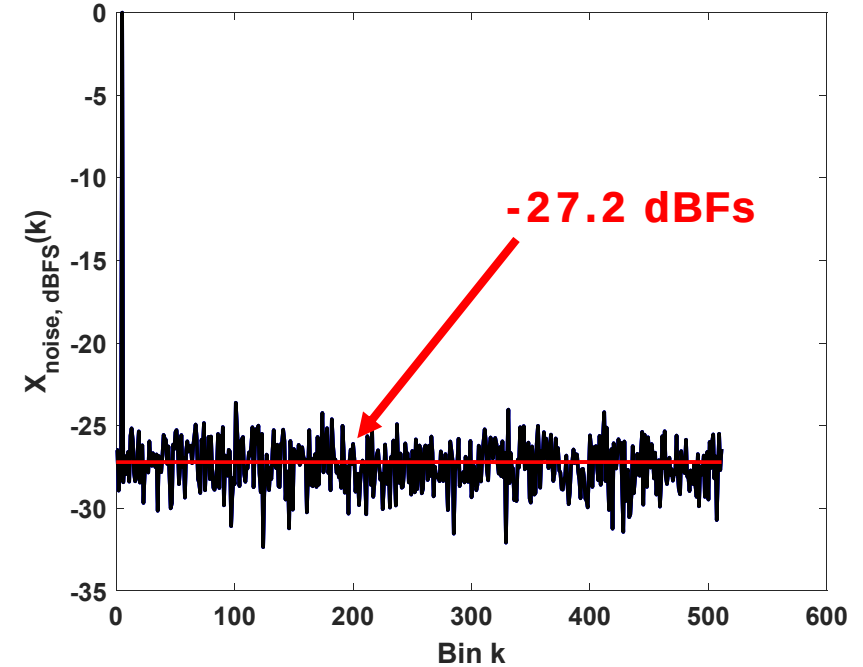
10 times 1024 samples used to average the spectra



Averaged Spectra of the 10 realizations



Red: Average Bin value of the Averaged 10 spectras.



Exercise: Calculate the noise power, based on the noise-floor in dBFS.

Two Kind of Spectral Averaging

1. Averaging of different realizations.
2. Averaging of the power per pin, over all bins.

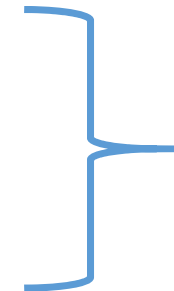


Does this make sense?

→ More later in the section about the CFAR Algorithmn.

Agenda

- o Review of the DFT and its properties
- o DFT Scaling
- o Sinewaves and Noise
- o Averaging
- o **Using the FFT in your project**
 - **Hardware**
 - **Software**
 - **Implementation Hints**



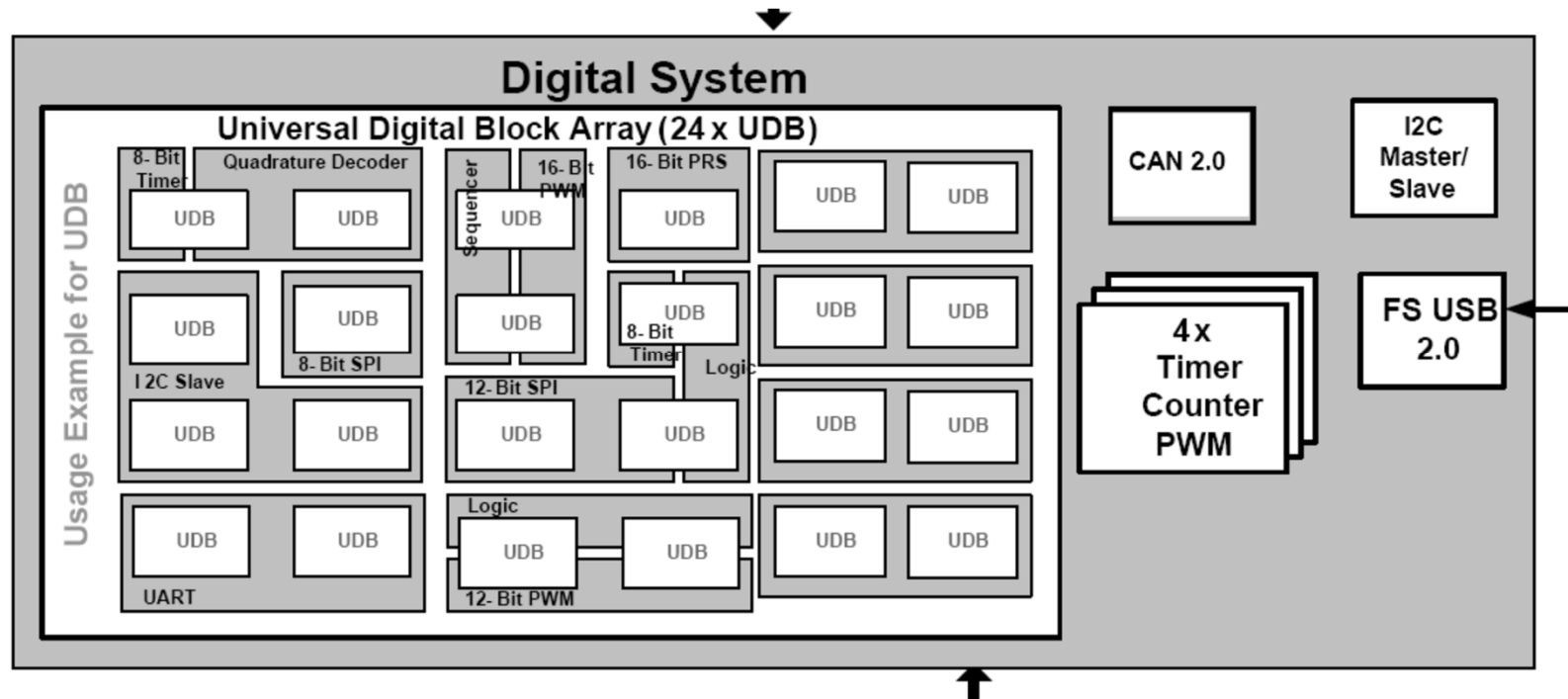
PSoC® 5LP: CY8C58LP

FFT with the PSoC® 5LP: CY8C58LP

- o Hardware
- o Software
- o Implementation Hints

Hardware FFT?

Look in the datasheet (PSoC® 5LP: CY8C58LP Family):



Unfortunately not possible. We are seeking for an software FFT.

Software FFT ?

- PSoC 5LP: 32-bit ARM Cortex-M3 core
- We can use **Cortex Microcontroller Software Interface Standard (CMSIS)**
<http://www.keil.com/pack/doc/CMSIS/General/html/index.html>

It provides DSP functionality:

DSP	All Cortex-M	DSP library collection with over 60 Functions for various data types: fixed-point (fractional q7, q15, q31) and single precision floating-point (32-bit). Implementations optimized for the SIMD instruction set are available for Cortex-M4/M7/M33/M35P.
------------	--------------	---

General Discription of the fft: See Documentation in the CMSIS Folder!

Read this
for your
project

► CMSIS ► Documentation

Name	Änderungsdatum	Typ
Core	13.09.2019 18:09	Dateiordner
Core_A	13.09.2019 18:09	Dateiordner
DAP	13.09.2019 18:09	Dateiordner
Driver	13.09.2019 18:09	Dateiordner
DSP	13.09.2019 18:09	Dateiordner
General	13.09.2019 18:09	Dateiordner
NN	13.09.2019 18:09	Dateiordner
Pack	13.09.2019 18:09	Dateiordner
RTOS	13.09.2019 18:09	Dateiordner
RTOS2	13.09.2019 18:09	Dateiordner
SVD	13.09.2019 18:09	Dateiordner
Zone	13.09.2019 18:09	Dateiordner
index.html	18.03.2019 15:46	Firefox HTML Doc

CMSIS-DSP

CMSIS DSP Software Library

Revision History of CMSIS-DSP

Deprecated List

▼ Reference

► Basic Math Functions

► Fast Math Functions

► Complex Math Functions

► Filtering Functions

► Matrix Functions

▼ Transform Functions

► Complex FFT Functions

► DCT Type IV Functions

► Real FFT Functions

► Controller Functions

► Statistics Functions

► Support Functions

► Interpolation Functions

► Examples

► Data Structures

► Data Fields

Description

The CMSIS DSP library includes specialized algorithms for computing the FFT of real data sequences. The FFT is defined over complex data but in many applications the input is real. Real FFT algorithms take advantage of the symmetry properties of the FFT and have a speed advantage over complex algorithms of the same length.

The Fast RFFT algorithm relies on the mixed radix CFFT that save processor usage.

The real length N forward FFT of a sequence is computed using the steps shown below.

Input

CFFT N/2

Split RFFT process

RFFT Output

Real Fast Fourier Transform

The real sequence is initially treated as if it were complex to perform a CFFT. Later, a processing stage reshapes the data to obtain half of the frequency spectrum in complex format. Except the first complex number that contains the two real numbers $X[0]$ and $X[N/2]$ all the data is complex. In other words, the first complex sample contains two real values packed.

The input for the inverse RFFT should keep the same format as the output of the forward RFFT. A first processing stage pre-process the data to later perform an inverse CFFT.

Input

Split RIFFT process

CFFT N/2

RIFFT Output

Real Inverse Fast Fourier Transform

The algorithms for floating-point, Q15, and Q31 data are slightly different and we describe each algorithm in turn.

Floating-point

The main functions are `arm_rfft_fast_f32()` and `arm_rfft_fast_init_f32()`. The older functions `arm_rfft_f32()` and `arm_rfft_init_f32()` have been deprecated but are still documented.

The FFT of a real N-point sequence has even symmetry in the frequency domain. The second half of the data equals the conjugate of the first half flipped in frequency. Looking at the data, we see that we can uniquely represent the FFT using only N/2 complex numbers. These are packed into the output array in alternating real and imaginary components:

$$X = \{ \text{real}[0], \text{imag}[0], \text{real}[1], \text{imag}[1], \text{real}[2], \text{imag}[2] \dots \text{real}[(N/2)-1], \text{imag}[(N/2)-1] \}$$

It happens that the first complex number (`real[0]`, `imag[0]`) is actually all real. `real[0]` represents the DC offset, and `imag[0]` should be 0. (`real[1]`, `imag[1]`) is the fundamental frequency, (`real[2]`, `imag[2]`) is the first harmonic and so on.

The real FFT functions pack the frequency domain data in this fashion. The forward transform outputs the data in this form and the inverse transform expects input data in this form. The function always performs the needed bitreversal so that the input and output data is always in normal order. The functions support lengths of [32, 64, 128, ..., 4096] samples.

Q15 and Q31

The real algorithms are defined in a similar manner and utilize N/2 complex transforms behind the scenes.

CMSIS – DSP Components: FFT - Documentation

**Hints for
Your
Project**

In Detail we use:

1. Real FFT
2. Fixed point numbers: q31
3. Command:
4. Nfft = 1024

```
void arm_rfft_q31 ( const arm_rfft_instance_q31 * S,  
                   q31_t * pSrc,  
                   q31_t * pDst  
                   )
```

CMSIS – DSP Component: FFT for your Project



Hints for Your Project

Some Pitfalls for implementation:

- Set the proper switches for the compiler, in order only to initialize the used FFT coefficients. Otherwise your EEPROM is too small.
- Include `arm_math.h`
- Manually include the necessary header files from CMSIS
- Check the **Installing_CMSIS** folder in Moodle for screenshots and documentation.

Sum-Up

- DFT calculates the frequencies components at discrete frequencies out of a series of samples.
- The discrete frequency spacing (resolution or bin frequency) is f_s/N

Literature

- "Digital Signal Processing ", Prentice Hall, Proakis and Manolakis, p285 → Properties of the Spectra
- "Signalübertragung", Springer, Ohm Lüke, p.35 → Properties of real signals
- "Digitale Signalverarbeitung", Kammeyer und Kroschel, Kap. 7 Die DFT -> Herleitung und Eigenschaften